

Simulation and Design of PID Controller for Quadcopter

From Equations in MATLAB to Workflows in Simulink, Testing and Simulating a PID Controller for a Quadcopter.

Justin Chee

University of Bristol, Independent Research
Bristol, UK/Singapore, Singapore

Student Email: yx25498@bristol.ac.uk

Personal Email: mrjustinchee@gmail.com

LinkedIn: www.linkedin.com/in/justin-chee-tc

Tools: MATLAB, Simulink

Abstract

Quadcopters are widely used in commercial drones for functions like aerial photography and delivery. A core aspect of the design of these drones is attitude stabilization, which is commonly addressed through a PID feedback loop in the flight controller. This study presents the design and validation of a PID-based flight controller for a quadcopter, developed in a MATLAB/Simulink simulation. A nonlinear dynamic model was derived from first-principles Newton-Euler equations and used as a plant for PID controllers for pitch, roll and yaw. The PID controllers were tuned using the Simulink PID Tuner, before being manually adjusted further to reduce the oscillations created by aggressive controller gains. This was then validated through step response, disturbance rejection and parameter uncertainty testing. The tuned controller then achieved an average of 13.49% overshoot and 4.45s settling time across the 3 axes, with a recovery from injected disturbances, demonstrating a viable attitude control design for future hardware implementation.

1. INTRODUCTION

Quadcopters are often described as unstable machines that, without the proper tuning and calibration, can allow for small errors to compound into significant deviations without the proper feedback control. As such, attitude stabilization is an issue that must be tackled first when designing and building one. To fix this, a common solution that remains at the forefront of industries is the use of Proportional, Integral and Derivative (PID) control systems. This is due also to the simplicity and wide understanding of PID, which is the reason that PID was chosen as a tuning method for this paper.

Instead of developing this on hardware on an actual drone however, this study looks at a simulation-based PID control workflow, validating the controller against a simulated dynamic model, which is commonly used in developing flight controllers before actual flight. This allows for the testing of conditions like gusts, which can be modelled as external disturbances that are otherwise difficult to test for on physical hardware. The simulation therefore serves as a first

step towards the eventual goal of building a fully functional hardware quadcopter.

2. METHODOLOGY

1. System Modelling

The quadcopter was modelled as a rigid body with 6 degrees of freedom. The thrust produced by the four rotors acting along the vertical (Z) axis of the body was modelled into the standard Euler rotational matrix (Wolfram, 2026), seen in Equation 1.

$$R = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi) \quad [1]$$

Where R is the overall rotation matrix, $R_z(\psi)$ is the yaw rotation matrix, $R_y(\theta)$ is the pitch rotation matrix and $R_x(\phi)$ is the roll rotation matrix.

Due to the smaller angular rates used in this study, gyroscopic cross-coupling terms were insignificant compared to direct torque terms, which was the leading factor in deciding on the use of independent axes rather than a fully coupled system. Motor thrusts and torque of each propeller were modelled using standard quadratic equations in relation to rotor speeds, seen in Equation 2-4 (AccPhys, 2026). This then formed the basis of connecting command torques to individual rotor speeds subsequently.

$$I_{xx} \cdot p = (I_{yy} - I_{zz}) \cdot q \cdot r + \tau_\phi \quad [2]$$

$$I_{yy} \cdot q = (I_{zz} - I_{xx}) \cdot p \cdot r + \tau_\theta \quad [3]$$

$$I_{zz} \cdot r = (I_{xx} - I_{yy}) \cdot p \cdot q + \tau_\psi \quad [4]$$

The physical parameters that were used in the simulation are summarised below in Table 1.

Table 1: Physical Parameters in Simulation and with units.

m	0.033 kg	I_{xx}	1.395e-5 kgm ²
g	9.81 kgms ⁻²	I_{yy}	1.436e-5 kgm ²
L	0.046 m	I_{zz}	2.173e-5 kgm ²
k	1.15e-7 N/(rad/s) ²	b	7.94e-12 Nm/(rad/s) ²

Where m is the mass of the controller, g is gravitational acceleration, L is the length of the drone arm, $I_{xx}/I_{yy}/I_{zz}$ are the second moments of area about the different axis, k is the thrust coefficient and b is the drag/yaw/torque coefficient.

II. Controller Design

Each rotational axis was controlled and modelled independently by a PID loop, consistent with the decisions made in the earlier section. However, initial testing with single-degree-of-freedom PID controllers showed severe spikes in output torque following step changes to the reference angle, a phenomenon later realised to be known as derivative kick. This happens when there is a sudden change in the setpoint, causing the derivative term to produce an undesirable spike in the controller output (Patsnap, 2026). This issue was then resolved by switching to a 2-degree-of-freedom (2-DOF) PID controller with the derivative setpoint weight parameter (c) set to 0. This causes the derivative term to act only on the measured variable, thereby eliminating the derivative kick.

Roll, Pitch and Yaw controller gains were obtained using Simulink's PID Tuner, which linearized the plant directly from the Simulink model, and parameterized it into response-time and transient-behaviour, each adjustable through a slider. This was adjusted to prioritize a well-damped response with minimal overshoot, which was then translated into individual P, I and D values. The filter coefficient (N) was then reduced to eliminate small infinite oscillations caused by high frequency values passing through an insufficiently filtered derivative.

3. RESULTS

The pitch, roll and yaw values were set at 10° , 5° and 15° respectively, or 0.1745, 0.0873 and 0.2618 radians. The workflow was then run in Simulink, with settling time defined as time to remain within 2% of the command's final value, a default setting in Simulink. The outcome of this is shown in Fig 1, with the overshoot and settling time shown in Table 2.

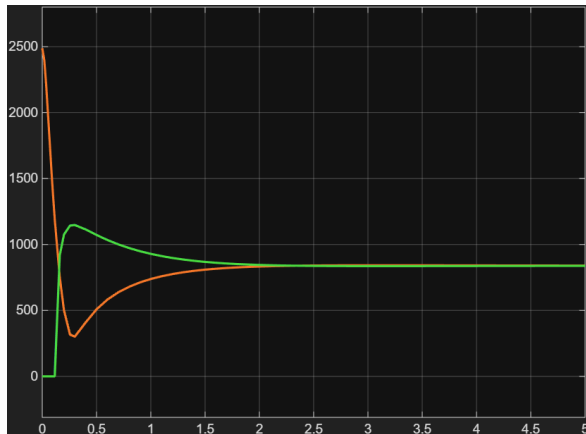


Figure 1: Motor Closed Loop Step Response (rad/s).

Table 2: Force Magnitudes with Different Cell Counts.

Metric	Roll	Pitch	Yaw
Overshoot (%)	13.225	13.2641	14.0027
Settling Time (s)	4.4796	4.4765	4.4183

From Fig 1, a clear convergence was observed on the base propeller rotational speed of 840 Rad/s which was consistent with the hover baseline. The single overshoot/undershoot

observed in the graphs also show a strong damping but not overdamping, consistent with what would be expected when one motor speeds up and another slows down to produce the same command torque.

A disturbance was then simulated, with a 0.00002 Nm torque disturbance injection, approximately 0.4% of the peak control torque after the system had reached a steady state. The angle deviated by 0.0188 rad, or 1.08° . The 0.5 second gust then required 2.7169 seconds to return to the commanded setpoint. The roll response can be observed in Fig 2.

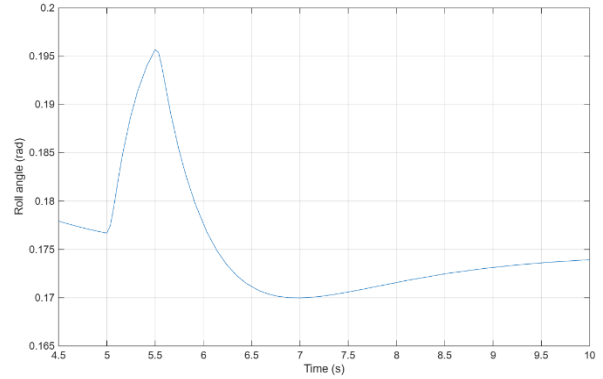


Figure 2: Roll Correction following Gust Disturbance.

From the graph, there is an obvious correction back to the commanded setpoint, which demonstrates that the integral term can be successful in eliminating steady-state errors produced by disturbances.

4. DISCUSSION

Discussion

The roll and pitch were adjusted and tuned using the PID tuner within Simulink without any issues, but yaw encountered some tuning challenges. Manual adjustments to the PID were required to eliminate oscillations produced by the torque and drag of the rotor on the yaw. This is expected, due to the dynamics of a quadcopter, with two axes unaffected by the rotor spins, highlighting the limitations of having a single-tuning configuration across all axes.

Limitations

Several further simplifications limited the reliability of the results, such as the decoupling of roll, pitch and yaw which hence neglected gyroscopic coupling, something that would have more significant effects as the manoeuvres get more aggressive. There was also no sensor noise validation performed, which in an actual drone would have affected measurement noise and potentially caused drift, ultimately degrading the drone's tracking performance. While these simplifications were reasonable for near-hover and small angle drifts, there would be a need for revisiting before carrying out more aggressive flight manoeuvres and implementing the software onto hardware.

5. WHAT I'D DO NEXT

This project demonstrates a simulation-based design and validation cycle for a quadcopter PID controller, with systematic tuning, modelling validation through a series of tests, while achieving effective disturbance rejection. The next steps would be to implement this onto a physical hardware quadcopter, and test theoretical physics against real world physics. This would allow factors like sensor noise, aerodynamic noise and modelling uncertainties to come to light, progressing towards increasingly aggressive flight manoeuvres.

6. BIBLIOGRAPHY

Euler angles from Wolfram MathWorld. Available at: <https://mathworld.wolfram.com/EulerAngles.html> (Accessed: 10 September 2026).

AccPhys *YouTube*. Available at: <https://www.youtube.com/watch?v=uhFYNHyaHs0> (Accessed: 10 September 2026).

Derivative kick problem in PID controllers and how to mitigate it, Patsnap Eureka. Available at: <https://eureka.patsnap.com/report-derivative-kick-problem-in-pid-controllers-and-how-to-mitigate-it> (Accessed: 15 September 2026).